Code559

Front-End Developer Course

MODULE 0

Setup

Install every tool once, so no later module stops for an install.

In this packet

1. **The lesson**: the complete Module 0 lesson, ready to print
2. **Setup log**: fill it in as you go, with checkpoint pictures
3. **Troubleshooting**: five problems that can stop you in Modules 0 and 1, and their fixes
4. **Self-assessment**: check you are ready for Module 1
5. **Answers**: to the lesson's questions, to check your own

SAMPLE FOR REVIEW

A 559 Solutions project

PART 1

The lesson

Install every tool once, so no later module stops for an install.

Time: about 60–90 minutes (estimate)

In this module you will learn:

- What each tool in a front-end developer's kit does, and why you need it
- How to show file extensions in File Explorer
- How to install VS Code, and how you will see your pages in Edge or Chrome (the Live Preview extension is optional)
- How to turn off VS Code's built-in AI features, and why this course asks you to
- How to install Git and set your name, email and default branch
- How to install the GitHub CLI and sign in to GitHub from the terminal
- How to install Node.js so you can run JavaScript outside the browser
- How to open a terminal in the right folder, and why you restart VS Code after each install

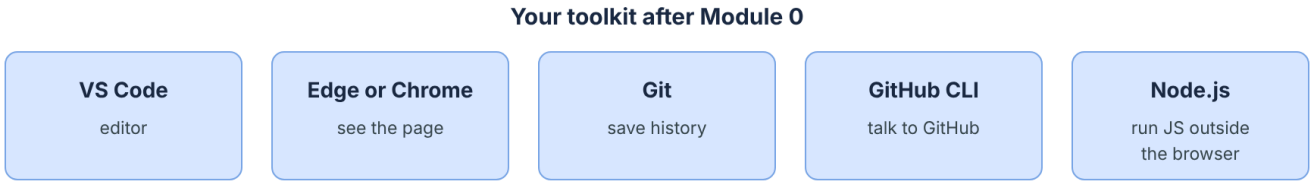
---

Why this matters

Every developer works inside a small set of tools. You write code in an editor, you look at the result in a browser, you save your history with Git, and you publish with GitHub. If one of those is missing in the middle of a lesson, you stop learning and start troubleshooting. This module puts everything in place first.

You will also make one decision that shapes the rest of the course: you will turn off the AI assistant in your editor. That is not because AI tools are bad. It is because this course asks you to explain every line you commit, and the simplest way to explain a line truthfully is to have written it yourself.

By the end you will have four tools installed and checked, a browser to see your pages, and a folder ready for the site you build in Module 1.



The five tools you use from now on and the job each one does. Edge is already on Windows 11; you install the other four in this module.

Before you start

1. You need a Windows 11 computer with an internet connection and permission to install software. This course is written for Windows only.
2. Create a free GitHub account at github.com if you do not have one. Pick a username you would be happy to show an employer; it appears in your website address later. Since March 2023, GitHub has required "all users who contribute code on GitHub.com" to turn on two-factor authentication (2FA): after your password, a second check such as a code from an app on your phone (GitHub Docs, *About mandatory two-factor authentication*). Turn it on when GitHub asks.
3. Decide where your course work will live. Choose a folder with a **short name and no spaces**, for example a folder called `code559` inside Documents. If OneDrive backs up your Documents folder (OneDrive settings, **Sync and backup**, **Manage backup**), choose a folder outside it instead, such as `C:\Users\you\code559`. The pictures in this course still show `Documents\code559`.

> **Checkpoint.** You can sign in to github.com, and you know the folder where your course work will live.

Core concepts

1. What is each tool for?

A front-end developer builds the part of a website that people see and use. To do that you need a place to write code, a way to see it, a way to keep its history, and a place to publish it.

VS Code is the editor where you type HTML, CSS and JavaScript. **Edge or Chrome** is the browser where you see your page; after each save you press F5 to reload it. **Git** keeps a history of your project as a series of saved snapshots, called commits. **GitHub** is a website that stores a copy of that history online and can publish your site. The **GitHub CLI** (`gh`) lets you do GitHub tasks from the terminal. **Node.js** runs JavaScript outside the browser; you will use it in the JavaScript modules.

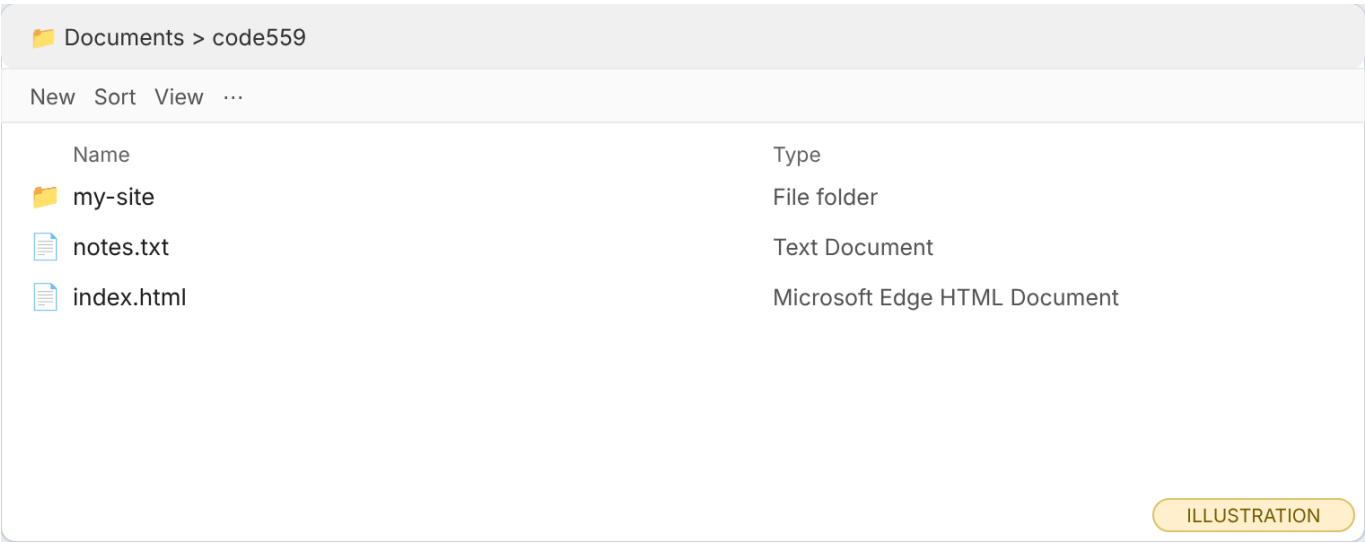
Look at the toolkit picture above. Every box is something you will use in almost every module.

Before you install anything, say out loud which tool you would use to (a) see your page, (b) save a snapshot, (c) publish. Check your answers against the paragraph above.

2. Why show file extensions?

A file extension is the part of a name after the last dot: `.html`, `.css`, `.js`. Windows hides it by default.

When extensions are hidden, a file you meant to name `index.html` can quietly become `index.html.txt`, and the browser treats it as plain text. Showing extensions stops that mistake before it happens.



With extensions shown, you can see that the file really ends in `.html`.

Open File Explorer. Select **View → Show → File name extensions** so it has a check mark.

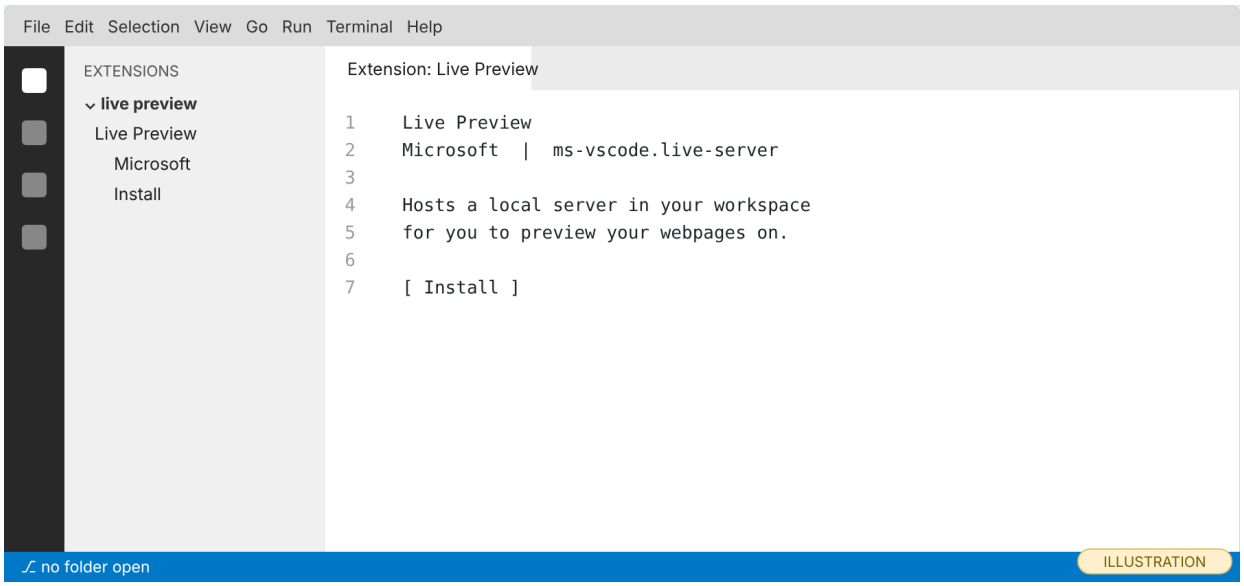
3. How will you see your pages?

VS Code is free and made by Microsoft. You write a page in VS Code and look at it in a browser. This course uses Edge or Chrome. Edge is already on Windows 11, so you need nothing extra.

From Module 1 on, you open each page straight in the browser: double-click the file in File Explorer, or drag it onto a browser window. (Double-clicking it in VS Code opens the code, not the page.) After each save, press **F5** to reload. Later modules also use **F12**, which opens the browser's DevTools. This works the same way on every page in the course.

Optional: Live Preview. Live Preview is a Microsoft extension for VS Code that, in its own words, "Hosts a local server in your workspace for you to preview your webpages on" (VS Code Marketplace, *Live Preview*). It shows your page inside VS Code and, by default, updates it as you type, even before

you save (its setting `livePreview.autoRefreshPreview` is "On All Changes in Editor" unless you change it). You do not need it for this course, and the Marketplace page says it "is still under development." If you install it, know one thing: VS Code opens a new folder in **Restricted Mode** (VS Code Docs, *Workspace Trust*), and Live Preview stays off in Restricted Mode. To turn it on, trust the folder: in the Restricted Mode banner at the top, click **Manage**, then click **Trust**. Only trust folders you made yourself, like your course folders.



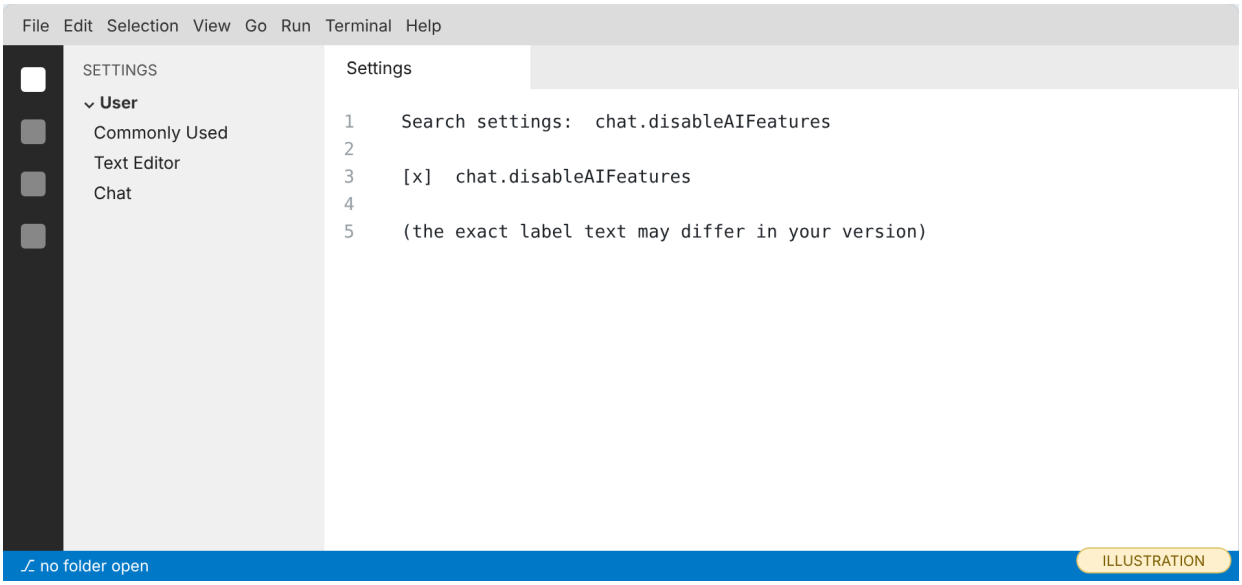
If you choose to install Live Preview, search the Extensions view for "Live Preview" and check that the publisher is Microsoft before you install.

You will install VS Code in the Exercise. Live Preview is step 5, and you may skip it.

4. Why turn off AI features?

Current VS Code includes GitHub Copilot features. VS Code's documentation says you can "disable the built-in AI features in VS Code with the `chat.disableAIFeatures`" setting (VS Code Docs, *Set up GitHub Copilot in VS Code*).

This course has one honesty rule: **if a tool wrote a line, you mark it and explain it.** While you are learning, it is much simpler to write every line yourself. You can turn AI features back on later, after you can read and explain code without help.



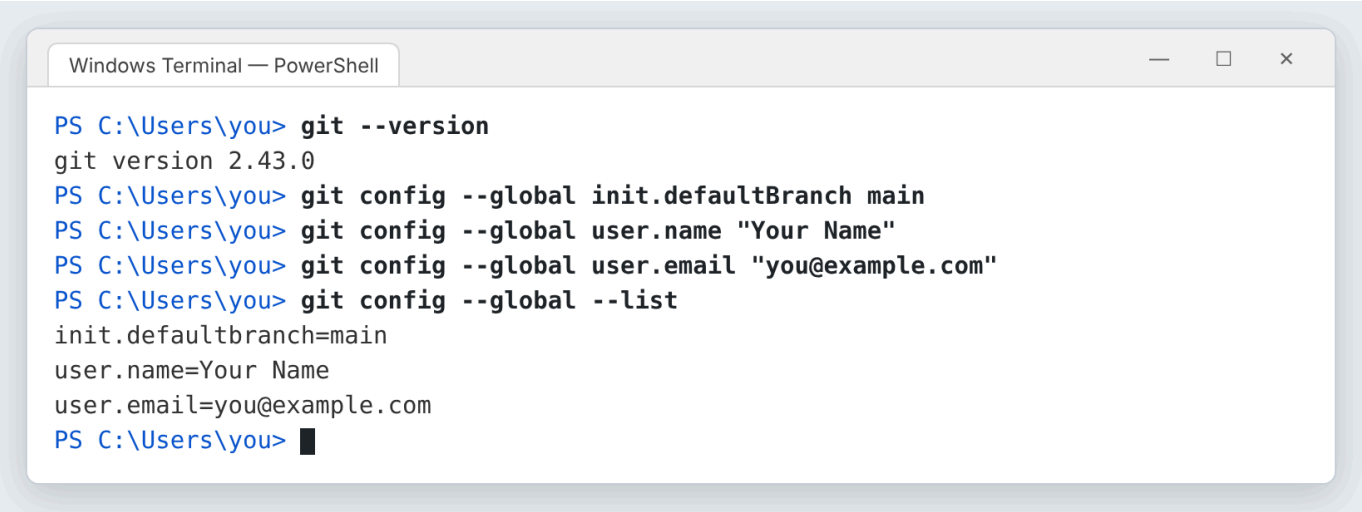
Type the setting name into the Settings search box and tick it.

You will do this in step 4 of the Exercise.

5. What does Git need to know about you?

Every commit records who made it. Git needs a name and an email for that.

You set three things once, with `git config --global`: your name, your email, and the name of the first branch in new projects. Git still names the first branch `master` unless you tell it otherwise; GitHub uses `main`. Setting `init.defaultBranch` to `main` keeps the two in agreement.



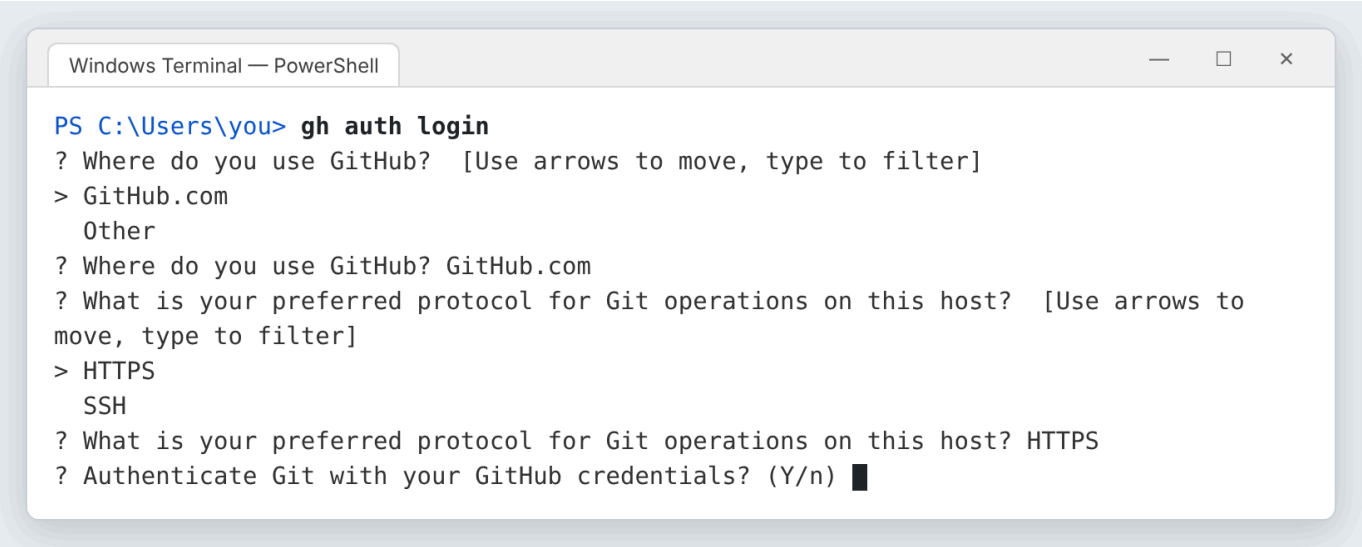
The three `config` commands print nothing when they work. `--list` proves they were saved. (real output, rendered; captured on Linux with Git 2.43.0. On Windows the version line ends in `.windows.1` or similar, and your version will be newer.)

Predict: what will `git config --global --list` show if you skip the email line? (Answer: only two lines.)

6. Why sign in to GitHub from the terminal?

The GitHub CLI can create a repository on GitHub and push your files to it with one command. It needs permission to act as you.

`gh auth login` asks a few questions, then opens your browser so you can approve the sign-in. You answer **GitHub.com**, then **HTTPS**, then **Yes** to "Authenticate Git with your GitHub credentials?", then choose **Login with a web browser**. Next, the terminal shows a one-time code on a line that starts `! First copy your one-time code:`. Copy the code or write it down. The terminal then says **Press Enter to open** a github.com address **in your browser**. Press Enter, and your browser opens a GitHub page. Paste or type the code there and approve the sign-in. The terminal then says **Authentication complete** and **Logged in as** your username. (These messages are checked against the GitHub CLI's source code in the cli/cli repository on GitHub.)



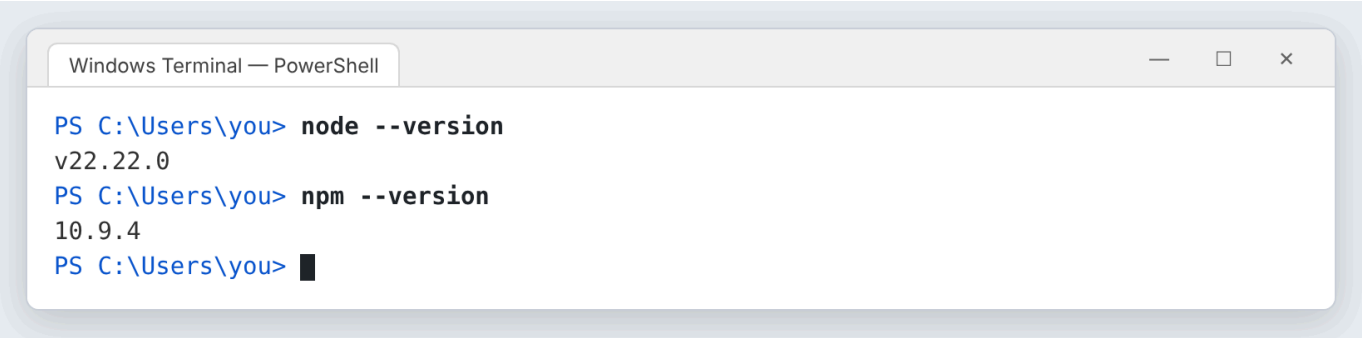
The first three prompts as GitHub CLI 2.89.0 prints them; the picture stops while `gh` waits for your answer to the third. For the first two, use the arrow keys to move and Enter to choose; for the (Y/n) question, press Enter to answer Yes. The "How would you like to authenticate" question and the one-time-code steps come next; they are described above, not shown here. (real output, rendered; captured on Linux)

After you sign in, `gh auth status` should say you are logged in to github.com. That is your proof.

7. Why install Node.js now?

Node.js runs JavaScript on your computer instead of inside a web page.

In Modules 5 to 8 you will practice the JavaScript language before you touch a web page. Node lets you run a script from the terminal and see the result. Installing it now means those modules never stop for setup. Choose the version that Node's download page marks **LTS** (Long-Term Support). When this post was checked, that was v24.21.0; Node's release schedule makes v26 the LTS version from 2026-10-28. Any LTS version, v24 or newer, works for this course.



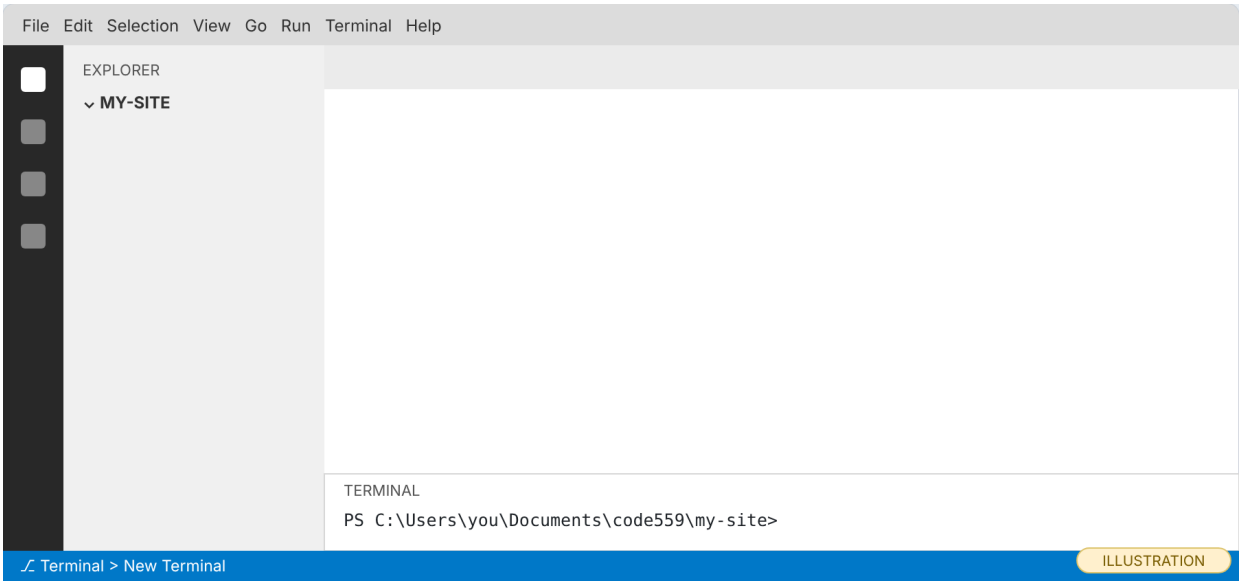
Two version numbers mean both Node and npm (its package manager) are installed. (real output, rendered; captured on Linux with Node 22. Your numbers will be newer, for example v24.x.)

After installing, run `node --version`. Any `v` followed by numbers is a pass.

Exercise

Your hands, about 60 minutes (estimate). Type every command. Do not paste. Typing is how your fingers learn the commands.

1. **Show file extensions.** In File Explorer: **View → Show → File name extensions**.
2. **Make your course folder.** Create the folder you chose (for example `Documents\code559`). Inside it, create a folder called `my-site`.
3. **Install VS Code.** Download it from code.visualstudio.com and run the installer. On the "Select Additional Tasks" screen, tick **Add "Open with Code" action to Windows Explorer file context menu** and **Add to PATH**. The wording on your screen may be a little different; look for the two options that add "Open with Code" to the right-click menu and add VS Code to PATH.
4. **Turn off AI features.** In VS Code: **File → Preferences → Settings**. Type `chat.disableAIFeatures` in the search box and tick it.
5. **Optional: install Live Preview.** You can skip this step: Edge or Chrome and F5 are all this course needs (Concept 3). To install it, click the Extensions icon in the left bar (or press Ctrl+Shift+X). Search **Live Preview**. Check the publisher says **Microsoft**. Click **Install**. It stays off until you trust your folder in step 6.
6. **Open a terminal.** In VS Code: **File → Open Folder...**, choose `my-site`, then **Terminal → New Terminal**. The prompt should end in `my-site>`. If VS Code shows a Restricted Mode banner or asks whether you trust this folder, trust it: it is your own folder (Concept 3). VS Code may ask again when you open each new practice folder in later modules; trust those too, for the same reason.



The prompt shows which folder the terminal is in. Always check it before you type a Git command.

7. **Install Git.** In the terminal type:

   ```
   winget install --id Git.Git -e --source winget
   ```

   (This is the command on git-scm.com's *Install for Windows* page.) When it finishes, **restart VS Code**: close it completely with **File → Exit**, then open VS Code again. It normally reopens your `my-site` folder; if not, use **File → Open Folder...**. Then choose **Terminal → New Terminal** and type `git --version`. A new terminal inside the same VS Code window is not enough, because it still uses the settings VS Code had before the install. VS Code's own docs say: "If the command isn't found, restart VS Code after installing Git" (VS Code Docs, *Quickstart: use source control in VS Code*). Still "not recognized"? Restart the computer, then try again.

8. **Configure Git.** Use your real name and the email on your GitHub account:

   ```
   git config --global init.defaultBranch main
   git config --global user.name "Your Name"
   git config --global user.email "you@example.com"
   git config --global --list
   ```

9. **Install the GitHub CLI.**

   ```
   winget install --id GitHub.cli --source winget
   ```

   (From the GitHub CLI manual's Windows install page.) Restart VS Code as in step 7 (**File → Exit**, open VS Code again, **Terminal → New Terminal**). Type `gh --version`, then `gh auth login` and answer as in Core Concept 6. Finish with `gh auth status`.

10. **Install Node.js (LTS).**

    ```
    winget install --id OpenJS.NodeJS.LTS -e --source winget
    ```

    Restart VS Code as in step 7 (**File → Exit**, open VS Code again, **Terminal → New Terminal**). Type `node --version` and `npm --version`. The package id `OpenJS.NodeJS.LTS` is listed in Microsoft's official winget catalog (the microsoft/winget-pkgs repository on GitHub). If winget says "No package found," download the LTS installer from nodejs.org instead.

11. **Break it on purpose.** In the same terminal, type `git stauts` (letters swapped). Read the message. Git says the command does not exist and suggests the closest real one.

    Git names the real command it thinks you meant. Learning to read messages like this is a skill you will use every day. (real output, rendered; captured on Linux with Git 2.43.0)

    Now try `git --verison`. This time Git only prints "unknown option" and its usage summary, with no suggestion. Same kind of typo, different help: always read the whole message.

Common misconception

> *"Git and GitHub are the same thing."*

What would you say? Hint: one of them works with no internet connection at all.

Summary

A front-end developer writes code in VS Code, sees it in Edge or Chrome (open the file, press F5 after each save; Live Preview is an optional extra), records its history with Git, and publishes it through GitHub, using the GitHub CLI from the terminal; Node.js runs JavaScript outside the browser. Showing file extensions stops files from getting a hidden `.txt` ending. AI features are off so that every line you

commit is a line you wrote and can explain. Git needs your name, your email and a default branch name of `main`. After every install, close VS Code completely and reopen it, then a new terminal and a version command prove the tool is ready.

Check your understanding

1. You install Git, open a new terminal in the same VS Code window, run `git --version` and get "not recognized." What is the most likely fix?
2. Why does this course set `init.defaultBranch` to `main`?
3. **What would you test** to prove the GitHub CLI is signed in?

Before next module

- [ ] File extensions are visible in File Explorer
- [ ] `git --version`, `gh --version` and `node --version` all print a version
- [ ] `gh auth status` says you are logged in
- [ ] AI features are off in VS Code
- [ ] You have a `my-site` folder and can open a terminal in it
- [ ] You know how to see a page: open it in Edge or Chrome and press F5 after each save (Live Preview is optional; if you installed it, your folder is trusted)

Go further

- VS Code Docs: *Setting up Visual Studio Code* and *Set up GitHub Copilot in VS Code* (the section on hiding AI features)
- git-scm.com: *Install for Windows*; *Pro Git*, chapter 1.6 "First-Time Git Setup"
- GitHub CLI manual: `gh auth login`
- nodejs.org: *Download Node.js*
- GitHub Docs: *Creating an account on GitHub*

What to keep

Keep a note of the four version numbers you saw (Git, GitHub CLI, Node, npm) and the folder path of `my-site`. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: git-scm.com "Install for Windows"; GitHub CLI "Installing gh on Windows" (github.com/cli/cli docs); VS Code Docs "Set up GitHub Copilot in VS Code"; VS Code Marketplace "Live Preview"; nodejs.org "Download Node.js". Checked 2026-10-11: VS Code Docs "Quickstart: use source control in VS Code"; VS Code Docs "Workspace Trust"; Live Preview's `package.json` (microsoft/vscode-livepreview on GitHub: untrusted workspaces not supported; `autoRefreshPreview` default); GitHub CLI 2.89.0 source code (cli/cli on GitHub: `internal/authflow/flow.go` and `pkg/cmd/auth/shared/login_flow.go`); microsoft/winget-pkgs on GitHub (`manifests/o/OpenJS/NodeJS/LTS`); GitHub Docs "About mandatory two-factor authentication" and "About two-factor authentication"; Microsoft Support "Back up your folders with OneDrive"; nodejs.org "Download Node.js" again (v24.21.0 LTS) and Node's release schedule (`schedule.json` in nodejs/Release on GitHub: v26 LTS from 2026-10-28).

PART 2

Setup log

Fill this in as you finish each step of the Exercise. Write the version number exactly as your terminal prints it. When every row is filled in and ticked, Module 0 is done.

Tool	Command that proves it	What you saw (version or result)	Done
File extensions	File Explorer → View → Show → File name extensions	Ticked? yes / no	☐
VS Code	Help → About		☐
AI features off	Settings → `chat.disableAIFeatures`	Ticked? yes / no	☐
Live Preview (optional)	Extensions view, publisher Microsoft	Installed? yes / skipped	☐
Git	`git --version`		☐
Git identity	`git config --global --list`	Name, email and `init.defaultbranch=main` shown? yes / no	☐
GitHub CLI	`gh --version`		☐
GitHub sign-in	`gh auth status`	Logged in as:	☐
Node.js	`node --version`		☐
npm	`npm --version`		☐

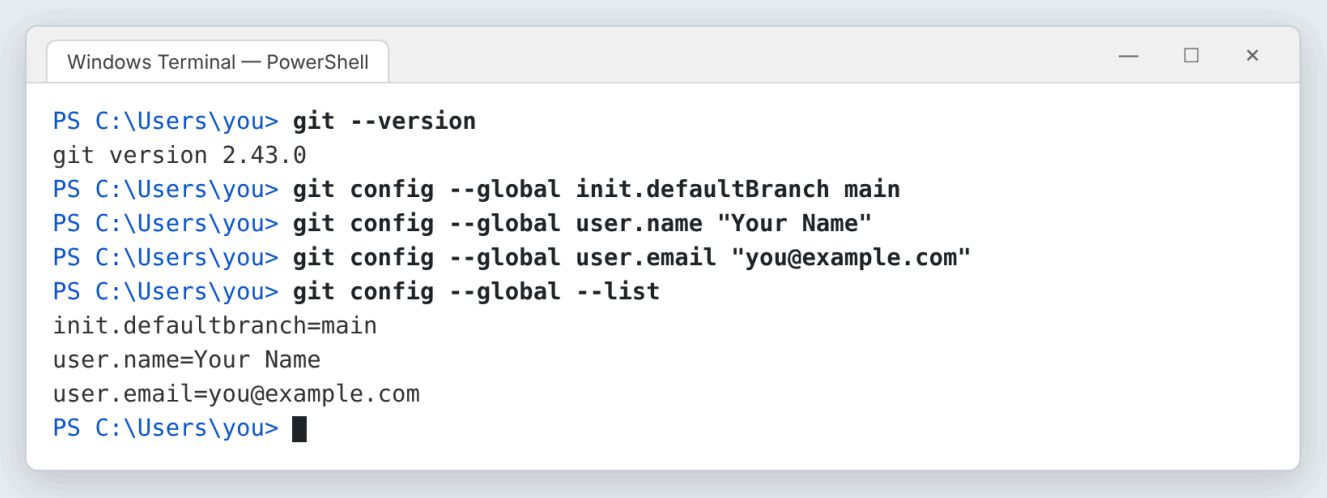
My course folder: _____

My GitHub username: _____

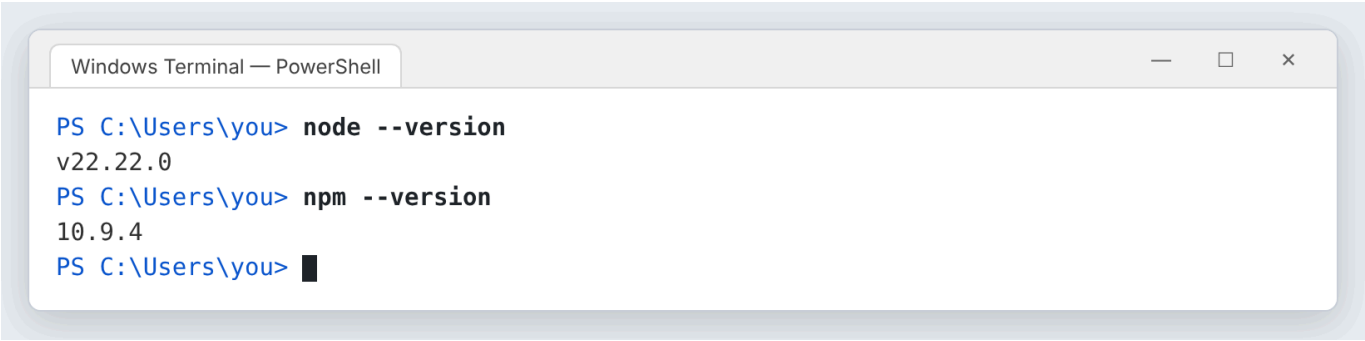
Date finished: _____

Checkpoints: what "done" looks like

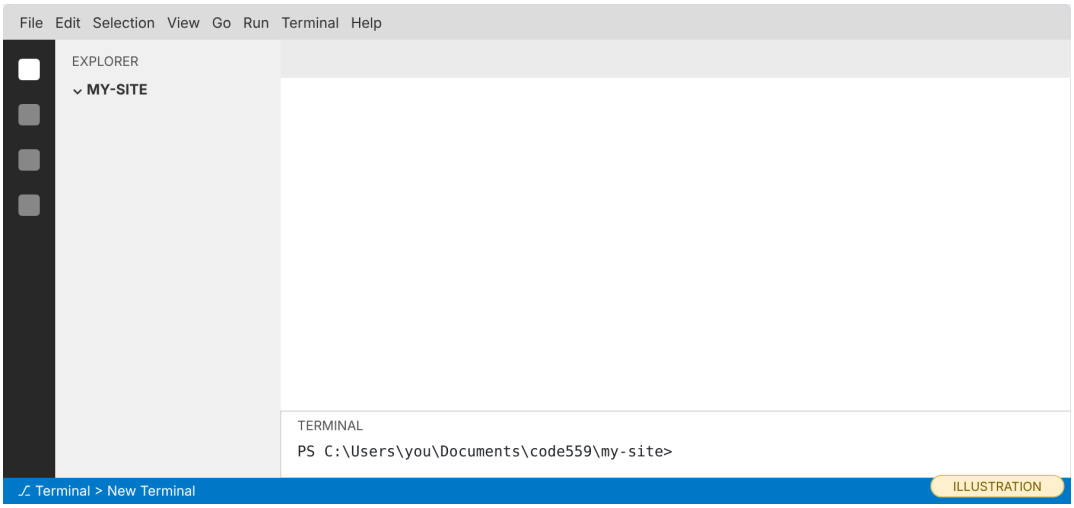
Compare your screen with these. Your version numbers will be different and newer; what matters is the shape of the result.



Git is set up when `--list` prints your default branch, name and email.



Node is ready when both commands print a version number.



Your terminal is in the right place when the prompt ends in `my-site>`.

PART 3

Troubleshooting: five problems that can stop you in Modules 0 and 1

Each problem has the symptom you see, why it happens, and the fix. Try the fix, then rerun the command from your setup log.

1. "not recognized" right after an install

Symptom. You installed Git, the GitHub CLI or Node, and typing `git --version` (or `gh` or `node`) gives an error saying the name is not recognized.

Why. A terminal reads the list of program locations (the PATH) when it opens, and a terminal inside VS Code gets that list from VS Code. VS Code itself started before the install, so it still has the old list.

Fix. Restart VS Code: close it completely with **File → Exit**, open it again, then choose **Terminal → New Terminal** and rerun the command. A new terminal inside the same VS Code window is not enough. Still not recognized? Restart the computer and try again. If it still fails, run the install command again and read its last lines for an error.

2. "winget" is not recognized

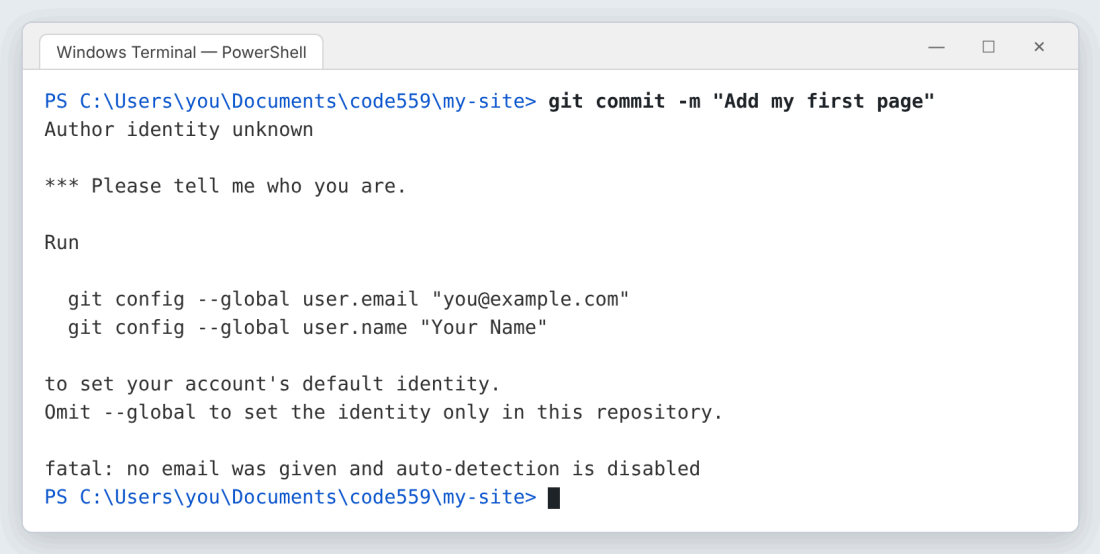
Symptom. The install commands fail because `winget` itself is not found.

Why. Microsoft's documentation says winget is part of the **App Installer**, delivered through the Microsoft Store, and that it "will not be available until you have logged into Windows as a user for the first time."

Fix. Open the Microsoft Store, search for **App Installer**, and install or update it. Then restart VS Code (**File → Exit**, open it again, **Terminal → New Terminal**) and type `winget --version`. If you cannot use the Store, download each tool from its own site instead: code.visualstudio.com, git-scm.com, cli.github.com and nodejs.org (choose LTS).

3. Git says "Author identity unknown"

Symptom. Your first `git commit` (in Module 1) stops with this message:



Git tells you exactly which two commands to run. The last line differs from computer to computer; often it starts `fatal: unable to auto-detect email address`. This picture was made with a setting that prints a shorter last line. real output, rendered; captured on Linux, with the folder path changed to the course folder

Why. Step 8 of the Exercise was skipped, or the email or name line had a typo.

Fix. Run all three lines of Exercise step 8 (default branch, name, email), and check them with `git config --global --list`. Then type `git status`. If it says `On branch master`, type `git branch -m main`: the repository was made before the default-branch line, and Module 1 step 15 looks for `main`. Commit again, then check that `git log --oneline` shows `HEAD -> main`.

4. The browser did not open during `gh auth login`

Symptom. The terminal shows a one-time code, but no browser window appears, or you closed it by mistake.

First check. `gh` waits for you: it says "Press Enter to open" a github.com address in your browser. If you have not pressed Enter yet, press it now.

Why. The sign-in happens on GitHub's website; the terminal only waits for it.

Fix. Open your browser yourself and go to **github.com/login/device**, the address GitHub Docs gives for entering device codes. Type the code from the terminal. GitHub Docs says these codes last 15 minutes, so if it has been longer, press Ctrl+C and run `gh auth login` again for a fresh code. Finish with `gh auth status`.

5. A file named `index.html` opens as plain text

Symptom. You double-click your page and see the code itself, or Windows calls it a "Text Document."

Why. Either you double-clicked it inside VS Code, which opens the code for editing, or, with file extensions hidden, the file was saved as `index.html.txt`.

Fix. Open the page from File Explorer (double-click it there, or drag it onto a browser window). If Windows still calls it a text document, turn on **View → Show → File name extensions** (Exercise step 1), rename the file so it ends in `.html` only, and accept the warning about changing the extension. In VS Code, always check the tab name after saving.

PART 4

Self-assessment

Rate yourself on each line. Aim for at least **3 (Proficient)** on every line before you start Module 1. If a line is a 1 or a 2, reread that Core Concept and redo its Exercise step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can say what each of the five tools is for	☐	☐	☐	☐
I can explain why file extensions should be visible	☐	☐	☐	☐
I can explain why AI features are off in this course	☐	☐	☐	☐
I can configure Git's name, email and default branch	☐	☐	☐	☐
I can sign in to GitHub from the terminal and prove it	☐	☐	☐	☐
I can prove a tool is installed with a version command	☐	☐	☐	☐
I can read an error message and find the hint in it	☐	☐	☐	☐

Three questions to answer in writing

1. Which step took you the longest, and what was the cause?

2. Pick one error message you saw. What did it tell you, and what did you do?

3. Explain to a friend, in two sentences, the difference between Git and GitHub.

PART 5

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson explains it.

Check your understanding

1. **"Not recognized" after installing Git.** Restart VS Code: close it completely with **File → Exit**, open it again, choose **Terminal → New Terminal**, and run `git --version` again. A new terminal in the same VS Code window still has the settings VS Code had before the install. Still "not recognized"? Restart the computer. (Exercise step 7)
2. **Why `init.defaultBranch` is `main`.** Git still names the first branch `master` unless you tell it otherwise, and GitHub uses `main`. Setting it to `main` keeps the two in agreement, so in Module 1 your branch has the name GitHub Pages looks for. (Core concept 5)
3. **What to test for the GitHub CLI.** Run `gh auth status`. It should say you are logged in to github.com. That is your proof. If it does not, run `gh auth login` again. (Core concept 6)

Common misconception

"Git and GitHub are the same thing." They are not. Git is a program on your computer that keeps a history of your project as commits; it works with no internet connection at all. GitHub is a website that stores a copy of that history online and can publish your site. (Core concept 1)